

# Project 4. RAG System

## 1. Overview

In this assignment, you'll build a Q&A web app that lets users ask questions about a set of documents using natural language. Your app will use **Retrieval-Augmented Generation (RAG)**: it will search a vector database of embedded document chunks, retrieve the most relevant ones, and pass them into a prompt for an OpenAI model to generate answers.

You'll use:

- **OpenAI** for embeddings and answering questions
  - **Chroma** for storing and retrieving embedded text chunks
  - **Gradio** for building a clean web interface
- 

## 2. Learning Objectives

By completing this project, you will be able to:

- Construct and use the basic architecture of a RAG pipeline
  - Generate embeddings using OpenAI's API
  - Store and query data with Chroma vector database
  - Build an interactive app with Gradio for users to ask questions
- 

## 3. Requirements

- You will use your OpenRouter API key to access Openai's embedding and language models.
  - A "requirements.txt" file is included with the assignment materials. Use this file to build a virtual environment that is in your project directory. You may need to add packages for document preprocessing, etc. You will need to activate this environment to run your code with the appropriate dependencies.
- 

## 4. Working Groups

- You may work in groups up to 2.

---

## What to Do

You will decide on the data that your AI will work with on the topic you want it to follow. This data will be meaningfully embedded into your vector database. You will need to extensively prompt your AI so it stays on track and follows directions, using only the sources given. While you can give it minimal instructions, it's likely to go off track if you do or be too rigid.

To complete this project, you will need to complete the following steps.

## 1. Document Loading & Chunking

To effectively chunk documents, you must first preprocess them. Documents may be stored in a variety of formats, including plain text, Markdown, PDF files, or Word documents. Before chunking, these files must be parsed to extract their textual content into a usable form (e.g., strings).

This preprocessing step is critical, as the quality and structure of the extracted text directly affect downstream tasks such as embedding generation, retrieval, and evaluation.

### Example: Working with PDF Files

PDFs are one of the most common document formats in academic and research settings, but they require specialized tools for text extraction. Several Python libraries are widely used for this purpose:

- **PyPDF2**
  - Simple and lightweight
  - Suitable for basic text extraction from digitally generated PDFs
  - May struggle with complex layouts or scanned documents
- **pdfplumber**
  - More robust than PyPDF2 for structured documents
  - Better handling of tables, columns, and layout-aware extraction
  - Often preferred for research papers and reports
- **PyMuPDF (fitz)**
  - High performance and versatile
  - Supports text, images, and layout extraction
  - Useful for large-scale or performance-sensitive pipelines
- **Unstructured (library) – see Free tier**
  - Designed for modern document pipelines (e.g., RAG systems)
  - Automatically handles multiple document types (including PDFs)
  - Provides structured outputs (sections, paragraphs, metadata)

When selecting a parsing library, consider:

- The complexity of your documents (e.g., multi-column layouts, figures)
- Whether the PDFs are scanned or digitally generated
- The importance of preserving structure (e.g., sections, headings)

## Document Selection and Chunking

1. You will need to source at least 3–5 documents (~10–25 pages total) on a topic of your choice. These documents should be:
  - Text-based (PDF, .txt, .md, or converted HTML)
  - Cohesive enough to ask meaningful questions about (e.g., not a list of tweets)
  - Publicly available or legally shareable, but could be class notes for example.
2. Split documents into manageable chunks (~500–800 tokens). However, don't just split them on say 500 token boundaries, instead:
  - If the text is structured--splitting along headers and major ideas is important! You would then sub-divide those splits further till you're within the 500 token boundary.
  - If your text is not structured, you should read it and decide what the best way to chunk is.
  - In both cases, you want to overlap between chunks because this helps preserve continuity. If an idea crosses those boundaries, the vector database would likely pick up both sections and the LLM would then have the text to work with in its response.
3. All of this should be done with code.

In summary:

- Source your content for the project
- Translate the source into text that can be used by python
- Determine how you will chunk the content

## 2. Embedding with OpenAI

To get the embeddings of your content (the vector representations), you will use ChromaDB to communicate with the OpenAI API. See our class session notes for code details:

```
from openai import OpenAI
import chromadb

from chromadb.utils import embedding_functions import OpenAIEmbeddingFunction
from dotenv import load_dotenv, find_dotenv

# Setup clients first, note we want chroma to be persistent
load_dotenv(find_dotenv(usecwd=True))
api_key = os.getenv("OPENROUTER_API_KEY")
if not api_key:
    print("Warning: OPENROUTER_API_KEY not found in environment")
    client = None
else:
    ai_client = OpenAI(api_key=api_key, base_url="https://openrouter.ai/api/v1")
    print("OpenRouter client created successfully!")

chroma_client = chromadb.PersistentClient(path='.chroma_persistent_db')
openrouter_client = OpenAI(api_key=os.getenv(), base_url="https://openrouter.ai/api/v1")

openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key=ai_client.api_key,
    # specify text-embedding-3-small
    model_name="text-embedding-3-small",
    # specify only 512 dimensions
    dimensions=512
)
```

```
# See if the collection exists, and if it doesn't, then load the data into it after you create it
```

```
try:
```

```
    collection = chroma_client.get_collection(name="my_document_collection",  
                                              embedding_function = openai_ef)
```

```
except:
```

```
    collection = chroma_client.create_collection(name="my_document_collection",  
                                                embedding_function=openai_ef)
```

```
# TODO: add content to this collection
```

Store the chunk text + metadata (like document title, heading titles, etc) alongside the vectors, because while Chroma will let you only store the text, you will want the metadata for each chunk of text as this will enable deeper functionality.

In summary, from the previous step, you have documents and their titles, so use these to:

- Chunk the text and create metadata for the text
- Store the text and metadata with each document in your Chroma collection

### 3. Store & Search with Chroma

In the prior step, you set up Chroma to use the OpenAI embedding model. You should have also stored the chunks with the metadata so that when you get results, you understand where they came from.

Now, when a user performs a query, you can use Chroma to perform similarity searches. Chroma will return the top  $k$  results, which you can then use for the next step. So in summary:

- Use Chroma to store the vectors and perform similarity searches.
- For a user query, find the top-k most similar document chunks.

### 4. Query Augmentation & LLM Response

Your final step in the RAG pipeline is to use the LLM. You should have an appropriate system prompt and keep your LLM on task! Don't let it answer queries that aren't related. As with the prior project, you must make sure the queries are safe before processing them, or inform the user you can't process the query. Thus, to complete this step, you will need to:

- Build a prompt using the retrieved chunks and the user's query.

- Send the augmented prompt to gpt-4o-mini
- Display the answer in the app.

## 5. Incorporate Memory

Your RAG+LLM pipeline should incorporate memory. This memory does not need to be external and can just be memory of the previous prompts. This way a person can ask about additional clarification on responses that the LLM gave previously.

## 6. Gradio Interface

The final step is to provide a Gradio interface for this project so that you have a decent UI to work with. You will provide a web interface to work with your API and should be able to:

- Enter a question
- View the answer and relevant source chunks

## 7. A 3-D Interactive Graph of Vector Embeddings

Provide an interactive visualization of the distances of returned results using plotly. This should give a preview of the chunk when hovering over it.

## 6. Submission Instructions

You will need to submit your project via Canvas in a folder named “proj4\_firstname\_lastname”. You must include:

- A README.md that contains instructions about your project. It should have the names of the people who worked on the project, including what they contributed. Instructions should let us know what file to run for the Gradio app.
- The source files to run your project without error.

**Due Date:** 6/3/26 by 11:59 PM

**Points:** 100

## Scoring

| Criteria                                                                                                                                                                                                    | Ratings                                                                                                                           | Pts    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------|
| <p>Is your code well-structured and executable?</p> <p>This means it's not just a long stream of code without any functions; it includes appropriate but judicious comments and executes without error.</p> | <p>20 pts - Full Marks</p> <p>0 pts - No Marks</p>                                                                                | 20 pts |
| <p>Did you index and embed your data chunks properly – including document id, document text, and metadata?</p>                                                                                              | <p>20 pts - Full Marks</p> <p>0 pts - No Marks</p>                                                                                | 20 pts |
| <p>Did you chunk your data to optimize retrieval results?</p> <p>This means separating on more than just token count boundaries and ensuring overlap between chunks.</p>                                    | <p>15 pts - Full Marks</p> <p>5 pts - The project only chunked on token counts or don't have overlap.</p> <p>0 pts - No Marks</p> | 15 pts |
| <p>Did you employ similarity search using Chroma to retrieve appropriate context?</p>                                                                                                                       | <p>10 pts - Full Marks</p> <p>0 pts - No Marks</p>                                                                                | 5 pts  |
| <p>Did you create a Gradio app to access, use, and test your RAG app?</p>                                                                                                                                   | <p>20 pts - Full Marks</p> <p>0 pts - No Marks</p>                                                                                | 10 pts |

| Criteria                                                                                                            | Ratings                                 | Pts    |
|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------|--------|
| Does your script support memory (history) as additional context in multi-turn interactions with the chat interface? | 10 pts – Full Marks<br>0 pts – No Marks | 15 pts |
| Does your prompt effectively restrict the RAG context?                                                              | 5 pts - Full Marks<br>0 pts - No Marks  | 5 pts  |
| Did you implement a 3-D embedding visualization?                                                                    | 10 pts - Full Marks<br>0 pts - No Marks | 10 pts |

Total Points: 100